HANGZHOU DIANZI UNIVERSITY

Faculty of Computer Science

Laboratory Work 1

Secure REST API Development with CI/CD and Real CVE Vulnerability Analysis

Variant number: CVE-2020-1938

Surname and Name: Zheng Junyi

HDU number: 23320127

Date: October 26, 2026

1. Purpose of the Work

The purpose of this work is to gain practical, end-to-end experience in building a secure backend application with automated vulnerability checking, and in analyzing and remediating a real-world vulnerability in a controlled environment. The work covers the full security lifecycle: designing protection against the OWASP Top 10 (notably A01 – Broken Access Control, A03 – Injection, A07 – Identification and Authentication Failures), integrating security tools into a CI/CD pipeline, reproducing a known CVE with Vulhub, identifying its root cause, and verifying the fix.

2. Choosing a Stack and Initializing the Project

A Python + Flask stack was chosen. The project is managed with pip inside a virtual environment; data is stored in SQLite through the SQLAlchemy ORM; authentication uses PyJWT and bcrypt. The source directory is initialised as a git repository. The project is linked to a public GitHub repository for the CI/CD part.

- Language: Python 3.13 + Flask 3.1.3; ORM: Flask-SQLAlchemy 3.1.1; JWT: PyJWT 2.15.1; password hashing: bcrypt 5.0.0.
- Package manager: pip (python -m pip). Dependencies are pinned in requirements.txt.
- Version control: git repository initialised locally and linked to GitHub (repository URL: https://github.com/zhengjunyi23-source/lab1-secure-api).

3. Developing a Functional API

The API exposes three endpoints, all implemented in app/routes.py:

Method	Endpoint	Auth	Description
POST	/auth/login	No	Authenticate with username + password; returns a signed JWT
GET	/api/data	Yes (JWT)	Returns the list of all users
GET	/api/me	Yes (JWT)	Returns the profile of the currently authenticated user

SQLite is used for simplicity. Demo users are seeded on first run (alice / bob / mallory). mallory deliberately stores an unsafe <script> string in her about field so the XSS protection can be demonstrated. The endpoint behaviour is verified by 7 pytest tests, all passing:

```
$ pytest -q
.  [100%]
7 passed in 8.55s
```

4. Implementing Basic Security Measures

4.1 Protection against SQL injection (SQLi)

All database access goes through the SQLAlchemy ORM, which always binds user input as bound parameters (prepared statements). No SQL string is assembled by concatenation. For example, the login lookup is:

```
user = User.query.filter_by(username=username).first()
```

The test test_sqli_parameterised confirms that a payload such as `' OR '1'='1' --` does not bypass authentication (the request is rejected with 401).

4.2 Protection against Cross-Site Scripting (XSS)

Every string returned to the client is passed through an output-encoding gate (app/security.py::xss_safe), which recursively HTML-escapes all string values using html.escape(..., quote=True). A stored payload <script>alert('XSS')</script> is therefore returned as <script>alert('XSS')</script> and cannot execute in a victim's browser. This is verified by test_xss_escaped_on_output.

4.3 Protection against Broken Authentication

- Passwords are never stored in plaintext — they are hashed with bcrypt (salted, adaptive) in User.set_password / check_password.
- On successful login a JWT (HS256) is issued (app/security.py::create_token) with sub, iat, exp and iss claims.
- A JWT-verification middleware @require_token protects every restricted endpoint and returns 401 without a valid Bearer token.
- Failed logins return a generic message, so an attacker cannot tell a wrong username from a wrong password (no user enumeration).

5. Setting Up a CI/CD Pipeline with Security Scanners

A GitHub Actions workflow .github/workflows/ci.yml runs on every push to main and on every pull request. It performs: (1) pytest functional tests; (2) SAST with Bandit, producing bandit-report.html; (3) SCA with Safety, auditing requirements.txt and producing safety-report.json; (4) uploading the reports as artifacts.

Local verification of the exact pipeline commands:

```
$ bandit -r app
No issues identified.
Total issues (by severity): Low 0, Medium 0, High 0
$ safety check -r requirements.txt
0 vulnerabilities reported
No known security vulnerabilities reported.
```

GitHub Actions run result: the last successful run (#2, commit 8f9a331) on main passed every step, including the SAST (Bandit) and SCA (Safety) scans, and uploaded the scanner reports as artifacts. Run link: https://github.com/zhengjunyi23-source/lab1-secure-api/actions/runs/40442950773

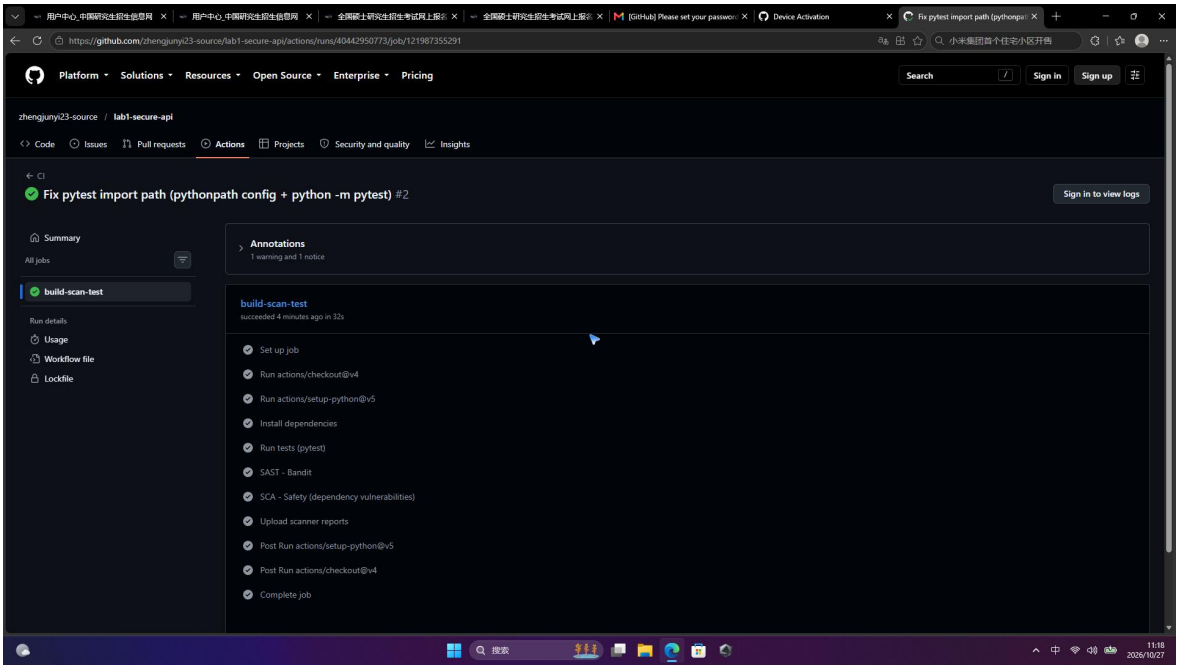


Figure 1. GitHub Actions job steps — tests, SAST (Bandit) and SCA (Safety) all green

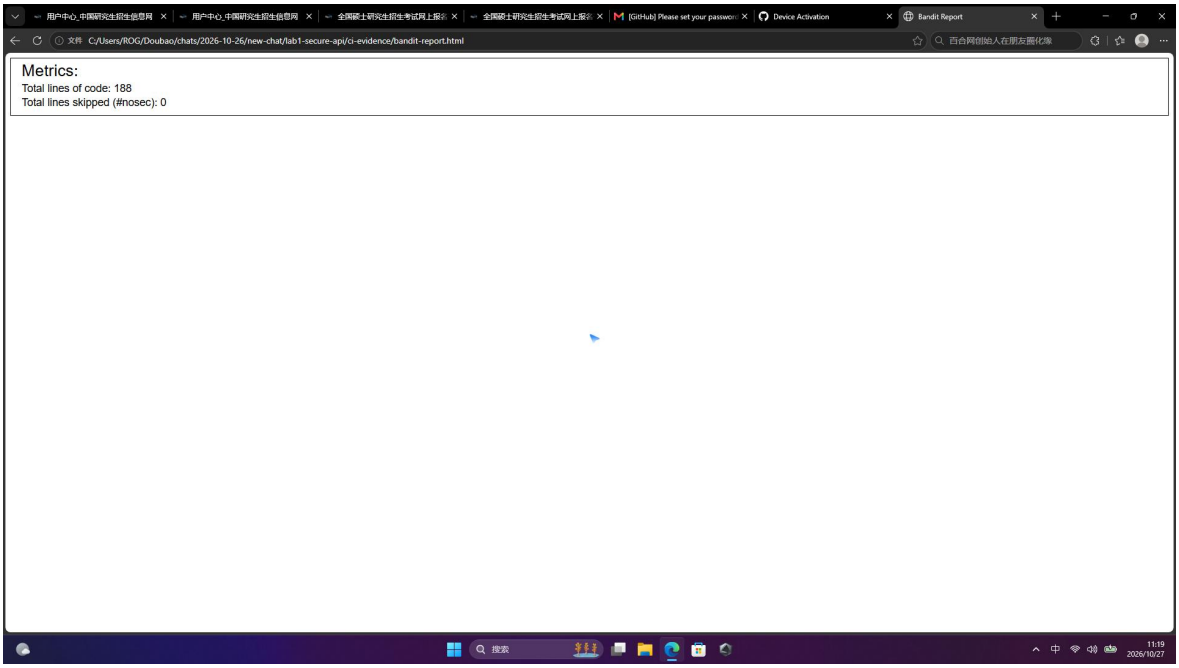


Figure 2. Bandit (SAST) report — 188 lines of code, 0 issues

6. Selecting and Preparing a Vulnerable Environment

The chosen vulnerability is CVE-2020-1938 (Apache Tomcat AJP “Ghostcat”). The Vulhub repository was cloned and the directory vulhub/tomcat/CVE-2020-1938 (Apache Tomcat 9.0.30) was used.

```
$ git clone https://github.com/vulhub/vulhub.git
$ cd vulhub/tomcat/CVE-2020-1938
```

7. Launching the Vulnerable Environment and Reproducing the Attack

The environment was started and both the HTTP (8080) and AJP (8009) ports were exposed:

```
$ docker compose up -d
NAMES  ...  PORTS
cve-2020-1938-tomcat-1  0.0.0.0:8009->8009/tcp, 0.0.0.0:8080->8080/tcp  Up
```

The Ghostcat PoC (a crafted AJP forward request) was then used to read the WEB-INF/web.xml file, which must never be reachable from outside. The AJP port 8009 was confirmed reachable and the request succeeded:

```
$ python poc.py 127.0.0.1 -p 8009 -f WEB-INF/web.xml
Getting resource at ajp13://127.0.0.1:8009/asdf
----------------------------
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee" ...>
  <display-name>Welcome to Tomcat</display-name>
  ...
</web-app>
```

This proves arbitrary file read without any authentication. The PoC sends an AjpForwardRequest carrying the attributes javax.servlet.include.path_info = <file> and servlet_path = /, which Tomcat's JspServlet honours and uses to read the requested file.

8. Analyzing the Root Cause of the Vulnerability

CVE-2020-1938 affects Apache Tomcat 6.x/7.x/8.x/9.x: because the AJP connector is exposed without authentication, an attacker can read or include arbitrary files inside the web application (e.g. source code, WEB-INF/web.xml) and, combined with a file-upload feature, potentially achieve remote code execution. The root cause is a configuration/input-handling weakness:

- The AJP/1.3 Connector is enabled by default on port 8009 (conf/server.xml).
- It is not bound to localhost (address not set) and has no secret / requiredSecret attribute — no authentication at all.
- Tomcat fully trusts the path, servlet and request attributes supplied by an AJP client, so a remote client can point JspServlet at any webapp file.

The vulnerable declaration in conf/server.xml:

```
<!-- Define an AJP 1.3 Connector on port 8009 -->
<Connector port="8009" protocol="AJP/1.3" redirectPort="8443" />
```

9. Developing and Applying Protection Measures

Because this deployment has no Apache front-end (mod_jk / mod_proxy_ajp), the AJP connector is not required. The fix therefore disables the AJP connector in conf/server.xml and stops exposing port 8009 in docker-compose.yml. The fixed files are provided in the cve-2020-1938-fix/ folder (server.xml, Dockerfile, docker-compose.yml, FIX-README.md).

```
$ docker compose down
$ docker compose up --build -d   # rebuild with hardened server.xml
```

Alternative remediations are discussed in the README (bind AJP to localhost and require a secret, or upgrade Tomcat to a patched version ≥ 9.0.31).

10. Verifying the Fix

- AJP port 8009 is no longer reachable: Test-NetConnection 127.0.0.1 -Port 8009 → False.
- Re-running the same PoC now fails: ConnectionRefusedError: [WinError 10061] (connection refused).
- Regression — the web application still works: GET http://127.0.0.1:8080/ returns HTTP 200 (Apache Tomcat welcome page).

These results demonstrate that the attack no longer succeeds while the main functionality remains intact.

11. Submissions (What Students Submit)

- Public git repository with the project code: https://github.com/zhengjunyi23-source/lab1-secure-api
- README.md in the repository (project & API description, security measures, scanner screenshots, CVE analysis and fix).
- Link to the last successful pipeline run: https://github.com/zhengjunyi23-source/lab1-secure-api/actions/runs/40442950773
- Folder with the fixes: cve-2020-1938-fix/ (server.xml, Dockerfile, docker-compose.yml).

Appendix A. Control Questions

1. Why is password hashing with bcrypt preferable to using SHA-256?

SHA-256 is a fast, general-purpose hash, so an attacker can brute-force or use rainbow tables on it quickly; bcrypt is deliberately slow (cost factor) and automatically salted, which makes offline cracking far more expensive and protects identical passwords from being stored identically.

2. Main difference between SAST and DAST?

SAST analyses source code statically (without running the app), finding issues early but with false positives; DAST tests the running application dynamically via real requests, finding exploitable issues but only in paths the scanner exercises.

3. How does a JWT token work, and how is it verified?

A JWT contains a header (algorithm), a payload (claims such as sub, exp, iss) and a signature. The server signs it with a secret (HS256) and, on protected requests, recomputes the signature and checks exp/iss; only a valid signature yields the payload.

4. Risks of not auditing third-party dependencies?

Known vulnerabilities in dependencies can be exploited (e.g. RCE, XSS), and you may unknowingly ship vulnerable versions. Auditing (SCA) detects and tracks these.

5. Advantages of Docker/Vulhub over installing vulnerable software directly?

Isolation (no harm to the host), easy setup/teardown, consistent versions, reproducible environments, and the ability to experiment safely with real CVE targets.

6. Difference between a vulnerability and an exploit; which were you reproducing?

A vulnerability is a weakness/defect in a system; an exploit is code that uses that weakness to cause an effect. In this work I reproduced an exploit of CVE-2020-1938 (reading an arbitrary file via AJP).

7. Why is regression testing critical after a patch?

A fix can break functionality that was previously working; regression testing confirms the attack is blocked while the main features still work.

8. How does reproducing known CVEs help a security specialist?

It builds practical knowledge of attack mechanics and indicators, supports detection and hardening, helps write accurate signatures/rules, and improves incident response and remediation skill.

Appendix B. References

[1] OWASP Foundation. OWASP Top 10 – 2021. https://owasp.org/www-project-top-ten/

[2] Auth0. Introduction to JSON Web Tokens. https://jwt.io/introduction

[3] Vulhub. Aapache Tomcat AJP Arbitrary File Read (CVE-2020-1938). https://github.com/vulhub/vulhub/tree/master/tomcat/CVE-2020-1938

[4] NVD. CVE-2020-1938 Detail. https://nvd.nist.gov/vuln/detail/CVE-2020-1938

[5] Apache Software Foundation. Announcement of the Ghostcat (AJP) fix. https://tomcat.apache.org/security-9.html

[6] Chaitin Tech. Ghostcat. https://www.chaitin.cn/en/ghostcat